MagicBox Installation Guide

Goal: Portainer reachable from the LAN at **https://magicbox.local:9443/**, with the `homelab` environment and the default stacks (RabbitMQ, PostgreSQL, Ollama, Grafana + Loki + Alloy). MagicBox has no fixed IP: it is always reached as `magicbox.local` (mDNS via Avahi). `magicbox$` = on the Mini PC, `controller$` = on your workstation in this repository. Replace `<...>` with your values.

1. Manual Steps (Mini PC)

1.1 Before you start

- [ ] Back up anything on the internal disk (the OS install erases it).
- [ ] Have: USB stick (4 GB+), Ethernet cable to the LAN (recommended) or the Wi-Fi name and password, router admin access.
- [ ] Controller and Mini PC on the same subnet (not guest Wi-Fi).

1.2 Firmware (UEFI)

Setup key is usually `Del`, boot menu `F7` (check the boot screen).

- [ ] Apply a GMKtec firmware update for your model, if one exists.
- [ ] Boot mode: UEFI. Secure Boot: enabled.
- [ ] *Restore on AC Power Loss*: **Power On**.
- [ ] Save and exit.

1.3 Install Ubuntu Server 24.04 LTS

(Also supported: Ubuntu 26.04, Debian 12/13.)

- [ ] Download the ISO from `https://ubuntu.com/download/server`, verify its SHA-256, write it to the USB stick, boot from it.
- [ ] Network: wired or Wi-Fi, DHCP (no static IP). Skipped Wi-Fi: see 1.3.1.
- [ ] Storage: entire internal disk, **no** passphrase encryption (it blocks unattended reboots).
- [ ] Server name: **`magicbox`**. Create your admin user (`<admin-user>`).
- [ ] Tick **Install OpenSSH server**. Featured snaps: **none**.
- [ ] Reboot, remove the USB stick, log in.

1.3.1 Wi-Fi (only if skipped during install)

Replace the three `<...>` values (`ip -br link` shows the Wi-Fi interface, usually starting with `wl`) and run:

```
magicbox$ sudo netplan set --origin-hint 60-wifi 'wifis.<wifi-interface>={dhcp4: true,
access-points: {"<wifi-name>": {password: "<wifi-password>"}}}'
magicbox$ sudo netplan apply
```

The first command writes `/etc/netplan/60-wifi.yaml` (readable by root only). If the name or password contains `'`, `"` or `\`, edit that file instead.

1.4 Update and collect facts

```
magicbox$ sudo apt update && sudo apt full-upgrade -y && sudo reboot
magicbox$ hostname                          # must print magicbox
magicbox$ cat /etc/os-release; uname -m     # supported OS, x86_64
magicbox$ ip -br link                       # LAN interface name
magicbox$ ip -br -4 addr                    # LAN subnet (e.g. 192.168.1.x/24)
```

1.5 Avahi (magicbox.local)

```
magicbox$ sudo apt install -y avahi-daemon avahi-utils libnss-mdns
magicbox$ sudo systemctl enable --now avahi-daemon
magicbox$ avahi-resolve-host-name -4 magicbox.local
```

- [ ] The last command prints `magicbox.local` and an IPv4 address.
- [ ] From the controller: `ping -c 3 magicbox.local` answers. (Linux controller: `sudo apt install avahi-daemon libnss-mdns` first.)

1.6 SSH key

```
controller$ mkdir -m 700 -p ~/.ssh/magicbox
controller$ ssh-keygen -t ed25519 -f ~/.ssh/magicbox/id_ed25519 -C magicbox
controller$ ssh-copy-id -i ~/.ssh/magicbox/id_ed25519.pub <admin-user>@magicbox.local
controller$ ssh -4 -i ~/.ssh/magicbox/id_ed25519 <admin-user>@magicbox.local 'sudo -v && echo
sudo-ok'
```

- [ ] Dedicated key for MagicBox: private `~/.ssh/magicbox/id_ed25519` (never share it), public `~/.ssh/magicbox/id_ed25519.pub`.
- [ ] Last command prints `sudo-ok`.
- [ ] Optional, after key login works (keep a session open while testing):

```
magicbox$ echo 'PasswordAuthentication no' | sudo tee /etc/ssh/sshd_config.d/10-magicbox.conf
magicbox$ sudo sshd -t && sudo systemctl reload ssh
```

1.7 Values for the inventory

Value	Source	Yours
ansible_host	always magicbox.local	magicbox.local
ansible_user	admin user	_____
ansible_ssh_private_key_file	key from 1.6	~/.ssh/magicbox/id_ed25519
ansible_ssh_extra_args	keep (IPv4 only)	"-4"
ansible_become_exe	Ubuntu 26.04 only	sudo.ws
magicbox_lan_interface	ip -br link	_____
magicbox_lan_cidr	LAN subnet, e.g. 192.168.1.0/24	_____

2. Controller setup

- [ ] Install `uv` (`brew install uv`), `git`, `make`. Windows: use WSL2.

```
controller$ make setup
controller$ make check
```

3. Configuration

```
controller$ cp inventory/hosts.example.yml inventory/hosts.yml
controller$ make vault-edit          # creates the encrypted vault on first use
```

- [ ] In `inventory/hosts.yml`: set `ansible_user`, `magicbox_lan_interface`, `magicbox_lan_cidr` (1.7). Keep `ansible_host: magicbox.local` and `ansible_ssh_private_key_file: ~/.ssh/magicbox/id_ed25519`.
- [ ] Ubuntu 26.04: uncomment `ansible_become_exe: sudo.ws`.
- [ ] In `vault.yml` (see `inventory/vault.example.yml`): `vault_become_password` (sudo password of `ansible_user`) and four different passwords of 12+ characters: `vault_portainer_admin_password`, `vault_postgresql_password`, `vault_rabbitmq_password`, `vault_grafana_admin_password`. Keep the vault password in your password manager.

```
controller$ make secrets-check
controller$ uv run ansible magicbox -m ansible.builtin.ping    # expect "pong"
```

4. Deployment

```
controller$ make deploy EXTRA="-e magicbox_confirm_destructive=true"   # first run
controller$ make deploy                                                # later runs
```

`make` asks only for the vault password; the sudo password comes from `vault_become_password`. The first run needs the confirmation flag because it enables the firewall. The CA certificate is saved to `artifacts/magicbox-local-ca.crt`. Portainer is initialised with the `homelab` environment.

Then pull the stack images on MagicBox (about 4 GB; shows progress and resumes after an interruption). Versions match `roles/stacks/defaults/main.yml`:

```
magicbox$ sudo docker pull rabbitmq:4.3.7-management
magicbox$ sudo docker pull postgres:18.6
magicbox$ sudo docker pull ollama/ollama:0.40.12
magicbox$ sudo docker pull grafana/grafana:13.2.5
magicbox$ sudo docker pull grafana/loki:3.7.9
magicbox$ sudo docker pull grafana/alloy:v1.21.2
```

- [ ] Each pull ends with `Status: Downloaded newer image` or `Image is up to date`.

Deploy the default stacks:

```
controller$ make stacks
```

Deploy only some stacks:

```
controller$ uv run ansible-playbook playbooks/stacks.yml --ask-vault-pass -e
'{"stacks_enabled": ["postgresql"]}'
```

5. Verification

```
controller$ make verify
controller$ make idempotency VAULT_PASSWORD_FILE=<file>
```

- [ ] `make verify` passes.
- [ ] `make idempotency` ends with "Idempotency check passed". Keep the password files outside the repo with mode 0600; delete them afterwards.

- [ ] From another LAN computer: `ping magicbox.local` works.
- [ ] Browser (CA trusted, section 6): `https://magicbox.local:9443/` opens with no warning; log in as `admin` with the vault password.
- [ ] `sudo reboot` on MagicBox: the page comes back, settings kept.
- [ ] `magicbox$ sudo ss -ltn | grep -E ':(9000|2375|2376) '` prints nothing.
- [ ] Portainer > Environments shows `homelab`; Stacks shows `rabbitmq`, `postgresql`, `ollama`, `observability`.
- [ ] `make status` reports every image and container as OK.
- [ ] A second `make stacks` reports `changed=0`.

Services (from the LAN):

Service	Address	Login
Grafana	https://magicbox.local:3000/	admin / vault_grafana_admin_password
RabbitMQ UI	http://magicbox.local:15672/	magicbox / vault_rabbitmq_password
RabbitMQ AMQP	amqp://magicbox.local:5672	same as UI
PostgreSQL	magicbox.local:5432, database magicbox	magicbox / vault_postgresql_password
Ollama	http://magicbox.local:11434/	none (LAN only)

- [ ] Grafana > Explore > Loki: `{container="postgresql"}` shows logs.

6. TLS trust (each client, once)

- [ ] Check the fingerprints match:

```
controller$ openssl x509 -in artifacts/magicbox-local-ca.crt -noout -fingerprint -sha256
magicbox$ sudo openssl x509 -in /opt/magicbox/portainer/certs/ca.crt -noout -fingerprint
-sha256
```

- [ ] Copy `magicbox-local-ca.crt` to the client and install it:

Client	Command / steps
macOS	sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain magicbox-local-ca.crt
Windows (admin PowerShell)	Import-Certificate -FilePath magicbox-local-ca.crt -CertStoreLocation Cert:\LocalMachine\Root
Ubuntu/Debian	sudo cp magicbox-local-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates
Firefox	Settings > Certificates > View Certificates > Authorities > Import > trust for websites
iOS	Install the profile, then General > About > Certificate Trust Settings > enable
Android	Settings > Security > Encryption & credentials > Install a certificate > CA certificate

- [ ] Restart the browser.

The CA only covers `magicbox.local` and `magicbox`. The server certificate is renewed automatically by any playbook run within 30 days of expiry.

6.1 If a client cannot resolve magicbox.local

- [ ] Client on the same LAN subnet (not guest Wi-Fi, not VPN, not another VLAN).
- [ ] macOS, iOS, Windows 10/11: built in. Android 12+: most browsers.
- [ ] Linux: `sudo apt install avahi-daemon libnss-mdns`.
- [ ] On MagicBox: `systemctl status avahi-daemon` is active.

7. Troubleshooting

Symptom	Fix
"SSH connection originates outside"	Controller uses IPv6: keep ansible_ssh_extra_args: "-4"; ssh -4 by hand
"Timeout waiting privilege escalation prompt"	Ubuntu 26.04 (sudo-rs): set ansible_become_exe: sudo.ws
"Unsupported OS"	Install Ubuntu Server 24.04 LTS
magicbox_lan_interface assertion	Use a name listed in the error
Admin password assertion	Create vault.yml; run with --ask-vault-pass
"ufw is inactive"	Add -e magicbox_confirm_destructive=true
Controller "outside" the LAN	Fix magicbox_lan_cidr or run from the LAN
Conflicting Docker packages	Check nothing uses them, then add the confirmation flag
Docker key fingerprint mismatch	Stop and verify Docker's key out of band
magicbox.local not resolving	systemctl status avahi-daemon; see 6.1
No network over Wi-Fi	Check name/password in /etc/netplan/60-wifi.yaml; sudo netplan --debug apply
Name became magicbox-2.local	Rename the other magicbox device; restart avahi-daemon
Certificate warning	Install the CA (section 6); restart the browser
Connection refused	sudo docker ps -a; sudo docker logs portainer; sudo ufw status
Vault password rejected at login	Portainer was already initialised: reset (8.4)
stacks: "Portainer rejected the vault password"	Password changed in the UI: update vault_portainer_admin_password
stacks: environment not found	Run make deploy first
make stacks waits a long time	make status (MISSING image: pull it, see 4)
Image pull stuck (Ollama)	sudo docker pull ollama/ollama:0.40.12 on MagicBox; df -h /var/lib
stacks: wait for containers times out	The error lists container states and logs; slow pull: re-run make stacks
Stack service rejects the vault password	Password was set at first start; change it in the service, then in the vault

8. Recovery

8.1 Repair drift

```
controller$ make deploy
```

8.2 Update packages

```
controller$ make update
controller$ make update EXTRA="-e os_update_reboot=true"   # if a reboot is reported
```

- [ ] Run monthly. The reboot stops Portainer and the stacks for a few minutes.

8.3 Firewall lockout

At the console: `sudo ufw disable`, fix `magicbox_lan_cidr` (IPv4 subnet), then redeploy with `-e magicbox_confirm_destructive=true`.

8.4 Lost admin password

```
magicbox$ sudo docker stop portainer
magicbox$ sudo docker run --rm -v portainer_data:/data portainer/helper-reset-password
magicbox$ sudo docker start portainer
```

8.5 Backup and restore

```
magicbox$ sudo docker stop portainer
magicbox$ sudo tar -C / -czf /root/magicbox-backup-$(date +%F).tgz opt/magicbox
var/lib/docker/volumes/portainer_data
magicbox$ sudo docker start portainer
```

Stack data lives in the volumes `postgresql_data`, `rabbitmq_data`, `grafana_data`, `loki_data` and

`ollama_data` under `/var/lib/docker/volumes/`. Stop the stack in Portainer before copying a volume. For PostgreSQL, prefer a logical dump:

```
magicbox$ sudo docker exec postgresql pg_dumpall -U magicbox | gzip > magicbox-pg-$(date +%F).sql.gz
```

The archive contains the CA key: store it encrypted, off the box. To restore, reinstall (sections 1-4), stop Portainer, run `sudo tar -C / -xzf <archive>`, and start Portainer.

8.6 Rotate the CA

Delete `/opt/magicbox/portainer/certs/` on MagicBox, run `make deploy`, and install the new CA on every client. Remove the old CA from the clients.